

INPUT AND OUTPUT

Handle approvals and user input

Surface Claude's approval requests and clarifying questions to users, then return their decisions to the SDK.

While working on a task, Claude sometimes needs to check in with users. It might need permission before deleting files, or need to ask which database to use for a new project. Your application needs to surface these requests to users so Claude can continue with their input.

Claude requests user input in two situations: when it needs **permission to use a tool** (like deleting files or running commands), and when it has **clarifying questions** (via the `AskUserQuestion` tool). Both trigger your `canUseTool` callback, which pauses execution until you return a response. This is different from normal conversation turns where Claude finishes and waits for your next message.

For clarifying questions, Claude generates the questions and options. Your role is to present them to users and return their selections. You can't add your own questions to this flow; if you need to ask users something yourself, do that separately in your application logic.

The callback can stay pending indefinitely. Execution remains paused until your callback returns, and the SDK only cancels the wait when the query itself is cancelled. If a user might take longer to respond than your process can reasonably stay running, return the `defer hook decision`, which lets the process exit and resume later from the persisted session.

This guide shows you how to detect each type of request and respond appropriately.

Detect when Claude needs input

Pass a `canUseTool` callback in your query options. The callback fires whenever Claude needs user input, receiving the tool name and input as arguments:

The callback fires in two cases:

1. **Tool needs approval:** Claude wants to use a tool that isn't auto-approved by permission rules or

modes. Check `tool_name` for the tool (e.g., `"Bash"` , `"Write"`).

2. **Claude asks a question:** Claude calls the `AskUserQuestion` tool. Check if `tool_name == "AskUserQuestion"` to handle it differently. If you specify a `tools` array, include `AskUserQuestion` for this to work. See [Handle clarifying questions](#) for details.

 To automatically allow or deny tools without prompting users, use [hooks](#) instead. Hooks execute before `canUseTool` and can allow, deny, or modify requests based on your own logic. You can also use the [PermissionRequest hook](#) to send external notifications (Slack, email, push) when Claude is waiting for approval.

Handle tool approval requests

Once you’ve passed a `canUseTool` callback in your query options, it fires when Claude wants to use a tool that isn’t auto-approved. Your callback receives three arguments:

Argument	Description
<code>toolName</code>	The name of the tool Claude wants to use (e.g., <code>"Bash"</code> , <code>"Write"</code> , <code>"Edit"</code>)
<code>input</code>	The parameters Claude is passing to the tool. Contents vary by tool.
<code>options</code> (TS) / <code>context</code> (Python)	Additional context including optional <code>suggestions</code> (proposed <code>PermissionUpdate</code> entries to avoid re-prompting) and a cancellation signal. In TypeScript, <code>signal</code> is an <code>AbortSignal</code> ; in Python, the <code>signal</code> field is reserved for future use. See ToolPermissionContext for Python.

The `input` object contains tool-specific parameters. Common examples:

Tool	Input fields
<code>Bash</code>	<code>command</code> , <code>description</code> , <code>timeout</code>
<code>Write</code>	<code>file_path</code> , <code>content</code>
<code>Edit</code>	<code>file_path</code> , <code>old_string</code> , <code>new_string</code>
<code>Read</code>	<code>file_path</code> , <code>offset</code> , <code>limit</code>

See the SDK reference for complete input schemas: [Python](#) | [TypeScript](#).

You can display this information to the user so they can decide whether to allow or reject the action, then return the appropriate response.

The following example asks Claude to create and delete a test file. When Claude attempts each operation, the callback prints the tool request to the terminal and prompts for y/n approval.

❗ In Python, `can_use_tool` requires [streaming mode](#) and a `PreToolUse` hook that returns `{"continue_": True}` to keep the stream open. Without this hook, the stream closes before the permission callback can be invoked.

This example uses a `y/n` flow where any input other than `y` is treated as a denial. In practice, you might build a richer UI that lets users modify the request, provide feedback, or redirect Claude entirely. See [Respond to tool requests](#) for all the ways you can respond.

Respond to tool requests

Your callback returns one of two response types:

Response	Python	TypeScript
Allow	<code>PermissionResultAllow(updated_input=...)</code>	<code>{ behavior: "allow", updatedInput }</code>
Deny	<code>PermissionResultDeny(message=...)</code>	<code>{ behavior: "deny", message }</code>

When allowing, pass the tool input (original or modified). When denying, provide a message explaining why. Claude sees this message and may adjust its approach.

Beyond allowing or denying, you can modify the tool's input or provide context that helps Claude adjust its approach:

- **Approve:** let the tool execute as Claude requested
- **Approve with changes:** modify the input before execution (e.g., sanitize paths, add constraints)
- **Approve and remember:** echo a suggested permission rule back so matching calls skip the prompt next time
- **Reject:** block the tool and tell Claude why
- **Suggest alternative:** block but guide Claude toward what the user wants instead
- **Redirect entirely:** use [streaming input](#) to send Claude a completely new instruction

Approve

```
async def handle_tool_request(tool_name, input_data, context):
    # Prompt user and return allow or deny
    ...

options = ClaudeAgentOptions(can_use_tool=handle_tool_request)
```

Approve with changes

Approve and remember

```
import asyncio

from claude_agent_sdk import ClaudeAgentOptions, ResultMessage, query
from claude_agent_sdk.types import (
    HookMatcher,
    PermissionResultAllow,
    PermissionResultDeny,
    ToolPermissionContext,
)

async def can_use_tool(
    tool_name: str, input_data: dict, context: ToolPermissionContext
) -> PermissionResultAllow | PermissionResultDeny:
    # Display the tool request
    print(f"\nTool: {tool_name}")
    if tool_name == "Bash":
        print(f"Command: {input_data.get('command')}")
        if input_data.get("description"):
            print(f"Description: {input_data.get('description')}")
    else:
        print(f"Input: {input_data}")

    # Get user approval
```

```

response = input("Allow this action? (y/n): ")

# Return allow or deny based on user's response
if response.lower() == "y":
    # Allow: tool executes with the original (or modified) input
    return PermissionResultAllow(updated_input=input_data)
else:
    # Deny: tool doesn't execute, Claude sees the message
    return PermissionResultDeny(message="User denied this action")

# Required workaround: dummy hook keeps the stream open for can_use_tool
async def dummy_hook(input_data, tool_use_id, context):
    return {"continue_": True}

async def prompt_stream():
    yield {
        "type": "user",
        "message": {
            "role": "user",
            "content": "Create a test file in /tmp and then delete it",
        },
    }

async def main():
    async for message in query(
        prompt=prompt_stream(),
        options=ClaudeAgentOptions(
            can_use_tool=can_use_tool,
            hooks={"PreToolUse": [HookMatcher(matcher=None, hooks=[dummy_hook])]},
        ),
    ):
        if isinstance(message, ResultMessage) and message.subtype == "success":
            print(message.result)

asyncio.run(main())

```

Reject

Suggest alternative

```
from claude_agent_sdk.types import PermissionResultAllow, PermissionResultDeny

# Allow the tool to execute
return PermissionResultAllow(updated_input=input_data)

# Block the tool
return PermissionResultDeny(message="User rejected this action")
```

Redirect entirely

Python

The user approves the action as-is. Pass through the `input` from your callback unchanged and the tool executes exactly as Claude requested.

Python TypeScript

```
async def can_use_tool(tool_name, input_data, context):
    print(f"Claude wants to use {tool_name}")
    approved = await ask_user("Allow this action?")

    if approved:
        return PermissionResultAllow(updated_input=input_data)
    return PermissionResultDeny(message="User declined")
```

TypeScript

```
async def can_use_tool(tool_name, input_data, context):
    print(f"Claude wants to use {tool_name}")
    approved = await ask_user("Allow this action?")

    if approved:
```

```
    return PermissionResultAllow(updated_input=input_data)
return PermissionResultDeny(message="User declined")
```

Python

TypeScript

The user approves but wants to modify the request first. You can change the input before the tool executes. Claude sees the result but isn't told you changed anything. Useful for sanitizing parameters, adding constraints, or scoping access.

Python TypeScript

```
async def can_use_tool(tool_name, input_data, context):
    if tool_name == "Bash":
        # User approved, but scope all commands to sandbox
        sandboxed_input = {**input_data}
        sandboxed_input["command"] = input_data["command"].replace(
            "/tmp", "/tmp/sandbox"
        )
        return PermissionResultAllow(updated_input=sandboxed_input)
    return PermissionResultAllow(updated_input=input_data)
```

Python

```
async def can_use_tool(tool_name, input_data, context):
    if tool_name == "Bash":
        # User approved, but scope all commands to sandbox
        sandboxed_input = {**input_data}
        sandboxed_input["command"] = input_data["command"].replace(
            "/tmp", "/tmp/sandbox"
        )
        return PermissionResultAllow(updated_input=sandboxed_input)
    return PermissionResultAllow(updated_input=input_data)
```

TypeScript

Python

The user approves and doesn't want to be asked again for this kind of call. The third callback argument carries `suggestions`, an array of ready-made `PermissionUpdate` entries. Echo one back in `updatedPermissions` to apply it. A suggestion with the `localSettings` destination writes the rule to `.claude/settings.local.json` so future sessions skip the prompt for matching calls. The Python example requires `claude-agent-sdk` 0.1.80 or later.


```
async def can_use_tool(tool_name, input_data, context):
    choice = await ask_user(f"Allow {tool_name}?", ["once", "always", "no"])

    if choice == "always":
        persist = [
            s for s in context.suggestions if s.destination == "localSettings"
        ]
        return PermissionResultAllow(
            updated_input=input_data, updated_permissions=persist
        )
    if choice == "once":
        return PermissionResultAllow(updated_input=input_data)
    return PermissionResultDeny(message="User declined")
```

TypeScript

```
async def can_use_tool(tool_name, input_data, context):
    choice = await ask_user(f"Allow {tool_name}?", ["once", "always", "no"])

    if choice == "always":
        persist = [
            s for s in context.suggestions if s.destination == "localSettings"
        ]
        return PermissionResultAllow(
            updated_input=input_data, updated_permissions=persist
        )
    if choice == "once":
        return PermissionResultAllow(updated_input=input_data)
    return PermissionResultDeny(message="User declined")
```

Python

TypeScript

The user doesn't want this action to happen. Block the tool and provide a message explaining why. Claude sees this message and may try a different approach.

Python TypeScript

```
async def can_use_tool(tool_name, input_data, context):
    approved = await ask_user(f"Allow {tool_name}?")

    if not approved:
        return PermissionResultDeny(message="User rejected this action")
    return PermissionResultAllow(updated_input=input_data)
```

```
async def can_use_tool(tool_name, input_data, context):
    approved = await ask_user(f"Allow {tool_name}?")

    if not approved:
        return PermissionResultDeny(message="User rejected this action")
    return PermissionResultAllow(updated_input=input_data)
```

The user doesn't want this specific action, but has a different idea. Block the tool and include guidance in your message. Claude will read this and decide how to proceed based on your feedback.

```
async def can_use_tool(tool_name, input_data, context):
    if tool_name == "Bash" and "rm" in input_data.get("command", ""):
        # User doesn't want to delete, suggest archiving instead
        return PermissionResultDeny(
            message="User doesn't want to delete files. They asked if you
could compress them into an archive instead."
        )
    return PermissionResultAllow(updated_input=input_data)
```

```
async def can_use_tool(tool_name, input_data, context):
    if tool_name == "Bash" and "rm" in input_data.get("command", ""):
        # User doesn't want to delete, suggest archiving instead
        return PermissionResultDeny(
            message="User doesn't want to delete files. They asked if you could
compress them into an archive instead."
        )
    return PermissionResultAllow(updated_input=input_data)
```

For a complete change of direction (not just a nudge), use [streaming input](#) to send Claude a new instruction directly. This bypasses the current tool request and gives Claude entirely new instructions to follow.

```
async for message in query(
    prompt="Analyze this codebase",
```

```
options=ClaudeAgentOptions(
    # Include AskUserQuestion in your tools list
    tools=["Read", "Glob", "Grep", "AskUserQuestion"],
    can_use_tool=can_use_tool,
),
):
    print(message)
```

```
async def can_use_tool(tool_name: str, input_data: dict, context):
    if tool_name == "AskUserQuestion":
        # Your implementation to collect answers from the user
        return await handle_clarifying_questions(input_data)
    # Handle other tools normally
    return await prompt_for_approval(tool_name, input_data)
```

```

return PermissionResultAllow(
    updated_input={
        "questions": input_data.get("questions", []),
        "answers": {
            "How should I format the output?": "Summary",
            "Which sections should I include?": ["Introduction", "Conclusion"],
        },
    }
)

```

```
import asyncio
```

```

from claude_agent_sdk import ClaudeAgentOptions, ResultMessage, query
from claude_agent_sdk.types import HookMatcher, PermissionResultAllow

```

```

def parse_response(response: str, options: list) -> str:
    """Parse user input as option number(s) or free text."""
    try:
        indices = [int(s.strip()) - 1 for s in response.split(",")]
        labels = [options[i]["label"] for i in indices if 0 <= i < len(options)]
        return ", ".join(labels) if labels else response
    except ValueError:
        return response

```

```

async def handle_ask_user_question(input_data: dict) -> PermissionResultAllow:
    """Display Claude's questions and collect user answers."""
    answers = {}

    for q in input_data.get("questions", []):
        print(f"\n{q['header']}: {q['question']}")

        options = q["options"]
        for i, opt in enumerate(options):
            print(f" {i + 1}. {opt['label']} - {opt['description']}")
        if q.get("multiSelect"):

```

```

        print(" (Enter numbers separated by commas, or type your own answer)")
    else:
        print(" (Enter a number, or type your own answer)")

    response = input("Your choice: ").strip()
    answers[q["question"]] = parse_response(response, options)

    return PermissionResultAllow(
        updated_input={
            "questions": input_data.get("questions", []),
            "answers": answers,
        }
    )

async def can_use_tool(
    tool_name: str, input_data: dict, context
) -> PermissionResultAllow:
    # Route AskUserQuestion to our question handler
    if tool_name == "AskUserQuestion":
        return await handle_ask_user_question(input_data)
    # Auto-approve other tools for this example
    return PermissionResultAllow(updated_input=input_data)

async def prompt_stream():
    yield {
        "type": "user",
        "message": {
            "role": "user",
            "content": "Help me decide on the tech stack for a new mobile app",
        },
    }

# Required workaround: dummy hook keeps the stream open for can_use_tool
async def dummy_hook(input_data, tool_use_id, context):
    return {"continue_": True}

async def main():
    async for message in query(
        prompt=prompt_stream(),
        options=ClaudeAgentOptions(
            can_use_tool=can_use_tool,
            hooks={"PreToolUse": [HookMatcher(matcher=None, hooks=[dummy_hook])]},
        ),
    ):
        if isinstance(message, ResultMessage) and message.subtype == "success":
            print(message.result)

```

```
asyncio.run(main())
```

Handle clarifying questions

When Claude needs more direction on a task with multiple valid approaches, it calls the `AskUserQuestion` tool. This triggers your `canUseTool` callback with `toolName` set to `AskUserQuestion`. The input contains Claude's questions as multiple-choice options, which you display to the user and return their selections.

💡 Clarifying questions are especially common in plan mode, where Claude explores the codebase and asks questions before proposing a plan. This makes plan mode ideal for interactive workflows where you want Claude to gather requirements before making changes.

The following steps show how to handle clarifying questions:

1 Pass a `canUseTool` callback

Pass a `canUseTool` callback in your query options. By default, `AskUserQuestion` is available. If you specify a `tools` array to restrict Claude's capabilities (for example, a read-only agent with only `Read`, `Glob`, and `Grep`), include `AskUserQuestion` in that array. Otherwise, Claude won't be able to ask clarifying questions:

2 Detect `AskUserQuestion`

In your callback, check if `toolName` equals `AskUserQuestion` to handle it differently from other tools:

3 Parse the question input

The input contains Claude's questions in a `questions` array. Each question has a `question` (the text to display), `options` (the choices), and `multiSelect` (whether multiple selections are allowed):

```
{
  "questions": [
    {
      "question": "How should I format the output?",
      "header": "Format",
      "options": [
        { "label": "Summary", "description": "Brief overview" },
        { "label": "Detailed", "description": "Full explanation" }
      ],
      "multiSelect": false
    },
    {
      "question": "Which sections should I include?",
      "header": "Sections",
      "options": [
        { "label": "Introduction", "description": "Opening context" },
        { "label": "Conclusion", "description": "Final summary" }
      ],
      "multiSelect": true
    }
  ]
}
```

See [Question format](#) for full field descriptions.

4 Collect answers from the user

Present the questions to the user and collect their selections. How you do this depends on your application: a terminal prompt, a web form, a mobile dialog, etc.

5 Return answers to Claude

Build the `answers` object as a record where each key is the `question` text and each value is the selected option's `label` :

From the question object	Use as
<code>question</code> field (e.g., <code>"How should I format the output?"</code>)	Key
Selected option's <code>label</code> field (e.g., <code>"Summary"</code>)	Value

For multi-select questions, pass an array of labels or join them with `" , "` . If you **support free-text input**, use the user's custom text as the value.

Question format

The input contains Claude's generated questions in a `questions` array. Each question has these fields:

Field	Description
<code>question</code>	The full question text to display
<code>header</code>	Short label for the question (max 12 characters)
<code>options</code>	Array of 2-4 choices, each with <code>label</code> and <code>description</code> . TypeScript: optionally <code>preview</code> (see <u>below</u>)
<code>multiSelect</code>	If <code>true</code> , users can select multiple options

The structure your callback receives:

```
{
  "questions": [
    {
      "question": "How should I format the output?",
      "header": "Format",
      "options": [
        { "label": "Summary", "description": "Brief overview of
key points" },
        { "label": "Detailed", "description": "Full explanation
with examples" }
      ],
      "multiSelect": false
    }
  ]
}
```

Option previews (TypeScript)

`toolConfig.askUserQuestion.previewFormat` adds a `preview` field to each option so your app can show a visual mockup alongside the label. Without this setting, Claude does not generate previews and the field is absent.

<code>previewFormat</code>	<code>preview</code> contains
<code>unset (default)</code>	Field is absent. Claude does not generate previews.
<code>"markdown"</code>	ASCII art and fenced code blocks
<code>"html"</code>	A styled <code><div></code> fragment (the SDK rejects <code><script></code> , <code><style></code> , and <code><!DOCTYPE></code> before your callback runs)

The format applies to all questions in the session. Claude includes `preview` on options where a visual comparison helps (layout choices, color schemes) and omits it where one wouldn't (yes/no confirmations, text-only choices). Check for `undefined` before rendering.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Help me choose a card layout",
  options: {
    toolConfig: {
      askUserQuestion: { previewFormat: "html" }
    },
    canUseTool: async (toolName, input) => {
      // input.questions[].options[].preview is an HTML string or
      undefined
      return { behavior: "allow", updatedInput: input };
    }
  }
})) {
  // ...
}
```

An option with an HTML preview:

```
{
  "label": "Compact",
  "description": "Title and metric value only",
  "preview": "<div style=\"padding:12px;border:1px solid
#ddd;border-radius:8px\"><div style=\"font-
size:12px;color:#666\">Active users</div><div style=\"font-
size:28px;font-weight:600\">1,284</div></div>"
}
```

Response format

Return an `answers` object mapping each question's `question` field to the selected option's `label` :

Field	Description
<code>questions</code>	Pass through the original questions array (required for tool processing)
<code>answers</code>	Object where keys are question text and values are selected labels
<code>response</code>	Optional freeform reply the user typed instead of answering the structured questions

For multi-select questions, pass an array of labels or join them with `" , "`. For per-question free text such as an “Other” option, put the user’s text in `answers[question]` as shown in [Support free-text input](#). Set `response` only when your UI lets the user dismiss the question card and type a general reply that isn’t an answer to any specific question. When `response` is set, Claude receives “The user responded: ...” instead of the per-question answer list.

```
{
  "questions": [
    // ...
  ],
  "answers": {
    "How should I format the output?": "Summary",
    "Which sections should I include?": ["Introduction",
"Conclusion"]
  }
}
```

Support free-text input

Claude’s predefined options won’t always cover what users want. To let users type their own answer:

- Display an additional “Other” choice after Claude’s options that accepts text input
- Use the user’s custom text as the answer value (not the word “Other”)

See the [complete example](#) below for a full implementation.

Complete example

Claude asks clarifying questions when it needs user input to proceed. For example, when asked to help decide on a tech stack for a mobile app, Claude might ask about cross-platform vs native, backend preferences, or target platforms. These questions help Claude make decisions that match the user’s preferences rather than guessing.

This example handles those questions in a terminal application. Here’s what happens at each step:

1. **Route the request:** The `canUseTool` callback checks if the tool name is `"AskUserQuestion"` and routes to a dedicated handler
2. **Display questions:** The handler loops through the `questions` array and prints each question with numbered options
3. **Collect input:** The user can enter a number to select an option, or type free text directly (e.g.,

“jquery”, “i don’t know”)

4. **Map answers:** The code checks if input is numeric (uses the option’s label) or free text (uses the text directly)
 5. **Return to Claude:** The response includes both the original `questions` array and the `answers` mapping
-

Limitations

- **Subagents:** `AskUserQuestion` is not currently available in subagents spawned via the Agent tool
- **Question limits:** each `AskUserQuestion` call supports 1-4 questions with 2-4 options each

Other ways to get user input

The `canUseTool` callback and `AskUserQuestion` tool cover most approval and clarification scenarios, but the SDK offers other ways to get input from users:

Streaming input

Use streaming input when you need to:

- **Interrupt the agent mid-task:** send a cancel signal or change direction while Claude is working
- **Provide additional context:** add information Claude needs without waiting for it to ask
- **Build chat interfaces:** let users send follow-up messages during long-running operations

Streaming input is ideal for conversational UIs where users interact with the agent throughout execution, not just at approval checkpoints.

Custom tools

Use custom tools when you need to:

- **Collect structured input:** build forms, wizards, or multi-step workflows that go beyond `AskUserQuestion`’s multiple-choice format
- **Integrate external approval systems:** connect to existing ticketing, workflow, or approval

platforms

- **Implement domain-specific interactions:** create tools tailored to your application's needs, like code review interfaces or deployment checklists

Custom tools give you full control over the interaction, but require more implementation work than using the built-in `canUseTool` callback.

Related resources

- **Configure permissions**: set up permission modes and rules
- **Control execution with hooks**: run custom code at key points in the agent lifecycle
- **TypeScript SDK reference**: full `canUseTool` API documentation